AGENTIC AI WORKSHOP · DAY 5 OF 10

From RAG to agentic RAG

Meridian reads Northstar's own policies, and learns to notice when a search came back wrong.

You build: **Reads policies** · about 3.8 h of material

Contents

1. Facts that change belong in documents, not in the model — Why this matters · 5 min
2. Embeddings: meaning as coordinates — Theory · 10 min
3. Chunking: the setting that silently decides quality — Theory · 8 min
4. Lab 1: Build the Northstar knowledge base — Lab · beginner · 30 min
5. Lab 2: Classic RAG, and how it fails — Lab · beginner · 20 min
6. Agentic RAG: grade, rewrite, retry, or give up — Theory · 10 min
7. Lab 3: Agentic RAG in LangGraph — Lab · intermediate · 35 min
8. Measure retrieval before you tune it — Theory · advanced · 10 min
9. Lab 4: Score your retriever — Lab · advanced · 20 min
10. Hybrid search, fusion and reranking — Theory · advanced · 10 min
11. Lab 5: Hybrid search, measured — Lab · advanced · 25 min
12. Grounded answers you can check — Theory · advanced · 8 min
13. Lab 6: Answers with verified citations — Lab · advanced · 25 min
14. RAG in production — Theory · advanced · 5 min
15. Check yourself — Quiz · 5 min
16. Day 5 recap — Recap · 3 min
17. Quiz answer key

WHY THIS MATTERS 5 MIN

Facts that change belong in documents, not in the model

Who approves a 12% limit increase? How long can it last? The answers live in Northstar's policies, and policies change.

No model knows Northstar Pay's internal rules. Retraining every time a policy changes is slow and can't cite its source. **Retrieval-augmented generation (RAG)** finds the right passage and gives it to the model, with a citation auditors can check.

The catch: retrieval can **silently** return the wrong passage, and the model will answer from it confidently. Today you'll see that happen, then fix it.

By the end of today you can

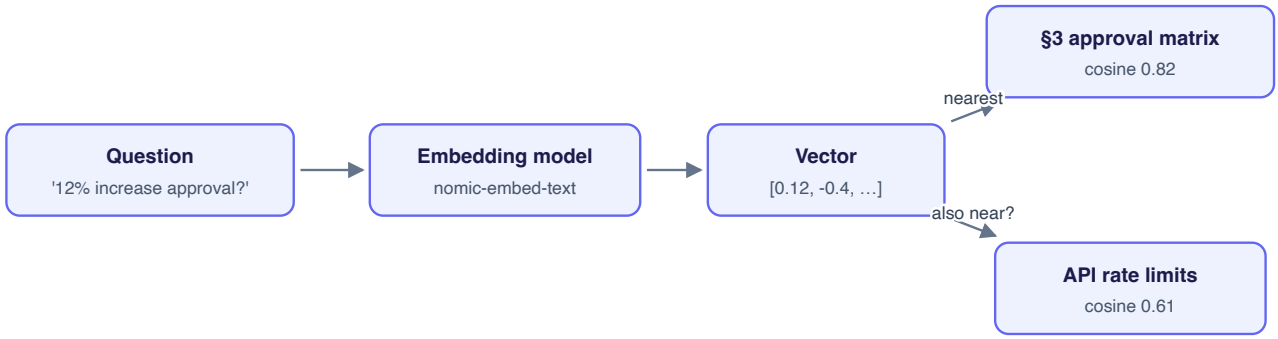
- Explain what an embedding captures, and what it misses
- Choose a chunk size and use metadata filters
- Build a vector store and a classic RAG chain
- Upgrade to agentic RAG: grade results, rewrite the query, or admit it can't find an answer
- Decide between RAG, fine-tuning and long context

THEORY 10 MIN

Embeddings: meaning as coordinates

An embedding model turns text into a vector so that texts with similar meaning land close together.

From text to nearest neighbours



1. The question is embedded with the SAME model used for the documents.
2. It becomes a point in a space of hundreds of dimensions.
3. The closest chunks (by cosine similarity) are retrieved: here, the approval matrix.
4. But 'limit' also appears in the API rate-limit doc. Similar words are not the same meaning: that's the trap.

embeddings_demo.py (optional: see the similarity numbers yourself)

```
"""Day 5 concept demo: what embeddings capture (and miss). Local Ollama embeddings; no API key needed.

Prereq: ollama pull nomic-embed-text.   Run: python embeddings_demo.py
"""
import os

import numpy as np
from langchain_ollama import OllamaEmbeddings

emb = OllamaEmbeddings(model=os.getenv("OLLAMA_EMBED_MODEL", "nomic-embed-text"))
TEXTS = ["temporary processing limit increase",
         "approval to raise a merchant's volume cap",
         "rolling reserve held on merchant payouts",
         "MCC 7995",
         "MCC 7999",
         "the cafeteria menu for Friday"]

if __name__ == "__main__":
    V = np.array(emb.embed_documents(TEXTS))
    V = V / np.linalg.norm(V, axis=1, keepdims=True)
    S = V @ V.T
    print("dims:", V.shape[1])
    for i, t in enumerate(TEXTS):
        print(f"{t[:42]:44}", " ".join(f"{x:5.2f}" for x in S[i]))
    print("\nNote rows 0/1 (same meaning, different words) vs 3/4 (different MCCs that look alike).")
```

Metric	Intuition	Note
Cosine	the angle between vectors	the default for text
Dot product	the same as cosine on normalised vectors	fastest
L2	straight-line distance	same ranking on normalised vectors

Where embeddings fail in payments: identifiers ("MCC 7995" vs "MCC 7999" look alike), negation ("not permitted" ≈ "permitted"), internal jargon ("four-eyes", "rolling reserve"). Fixes: metadata filters, keyword + vector (hybrid) search, re-ranking, and today's grading step.

THEORY 8 MIN

Chunking: the setting that silently decides quality

The retriever returns whole chunks. Too small loses context; too big buries the answer.

Chunk size	Upside	Downside
Small (100–300 tokens)	precise matches	"this limit": which limit?
Medium (400–800)	balanced	a good default for policies
Large (1000+)	full context	the right document, but the wrong paragraph

Split by structure first (Markdown headings), then by size. Keep tables (approval matrices, reserve %) in one piece.

Metadata is half the battle. Tag every chunk with `doc_type`, `section`, `effective_date`, and whether it's still current, so superseded policies can't answer, and every answer can cite *where it says that*.

LAB BEGINNER 30 MIN

Lab 1: Build the Northstar knowledge base

Goal. Index Northstar's policies in Chroma at three chunk sizes and compare what comes back.

1. Install and pull an embedding model

RUN

```
pip install "CHROMADB>=1.5,<2" "LANGCHAIN-CHROMA>=1.1,<2" langchain-text-splitters langchain-ollama
pyyaml
ollama pull nomic-embed-text
```

2. Create the docs/ folder

Save each document below as its own file in `docs/`. They're fictional, written with deliberate traps: a superseded policy, three different kinds of "limit", and numbers in tables.

3. Save docs/api-rate-limits.md

docs/api-rate-limits.md

```
---
doc_type: developer_guide
effective_date: 2026-01-01
owner: Platform Engineering
---
# Payments API Rate Limits
Each merchant API key is limited to 100 requests/second for payment creation and 20 requests/second
for refunds.
Exceeding the limit returns HTTP 429. Rate limits are unrelated to merchant processing limits.
Refunds are also limited to the original captured amount (no over-refunds).
```

4. Save docs/chargeback-and-disputes-policy.md

docs/chargeback-and-disputes-policy.md

```
---
doc_type: policy
effective_date: 2026-02-01
owner: Disputes
---
# Chargeback & Disputes Policy
## Monitoring thresholds (fictional internal thresholds, not real card-scheme values)
| Chargeback ratio (count, monthly) | Status | Action |
|---|---|---|
| below 0.65% | Normal | none |
| 0.65–0.9%   | Early warning | account manager review |
| above 0.9%  | Watchlist | no limit increases; remediation plan within 10 days |
| above 1.5%  | Excessive | rolling reserve raised to 15%; consider termination |
## Dispute handling
Evidence must be submitted within 20 calendar days of the dispute notification.
```

5. Save docs/merchant-risk-policy-2024-SUPERSEDED.md

docs/merchant-risk-policy-2024-SUPERSEDED.md

```
---
doc_type: policy
effective_date: 2024-01-01
superseded_by: merchant-risk-policy-2026
owner: Merchant Risk
```

```
---
# Merchant Risk Policy (v2024) — SUPERSEDED
## Temporary processing limit increases
Any increase up to 20% may be approved by the Risk Ops lead. Increases last up to 60 days.
```

6. Save docs/merchant-risk-policy-2026.md

docs/merchant-risk-policy-2026.md

```
---
doc_type: policy
effective_date: 2026-01-01
owner: Merchant Risk
---
# Merchant Risk Policy (v2026)

## 1. Processing limits
Each merchant has a monthly processing limit expressed in one currency. Month-to-date (MTD) volume is the sum of
all captured payments across every method (card, wallet, UPI), converted at the 16:00 London spot rate into the
limit currency.

## 2. Utilisation thresholds
- Above 90%: NEAR_LIMIT. The merchant's account manager must be notified within 1 business day.
- Above 100%: OVER_LIMIT. Risk Ops is notified immediately; new captures above the limit are queued for review.

## 3. Temporary processing limit increases
| Increase size (vs current limit) | Merchant risk tier | Approver |
|---|---|---|
| up to 5%  | Low or Medium | Risk Ops lead |
| 5–15%     | Low or Medium | Merchant risk officer (four-eyes) |
| 5–15%     | High          | Head of Merchant Risk |
| above 15% | any           | Risk Committee |
Increases last at most 30 calendar days and cannot be renewed back-to-back.
Merchants on the chargeback watchlist are not eligible for increases.
```

7. Save docs/pricing-sheet-2026-q4.md

docs/pricing-sheet-2026-q4.md

```
---
doc_type: pricing
effective_date: 2026-10-01
owner: Commercial
---
# Merchant Pricing Sheet — Q4 2026
| Method | Domestic MDR | Cross-border add-on |
|---|---|---|
| Debit card  | 0.90% | +1.00% |
| Credit card | 1.60% | +1.00% |
| Wallet      | 1.40% | +0.80% |
| UPI         | 0.00% | n/a |
FX conversion markup on settlement: 2.5% over mid-market. Chargeback fee: 15 USD per dispute.
```

8. Save docs/rolling-reserve-rules.md

docs/rolling-reserve-rules.md

```
---
doc_type: policy
effective_date: 2026-02-01
owner: Merchant Risk
---
```

```
# Rolling Reserve Rules
| Merchant risk tier | Reserve (% of daily settlement) | Hold period |
|---|---|---|
| Low    | 0%  | – |
| Medium | 5%  | 90 days |
| High   | 10% | 180 days |
High-risk MCCs (e.g. 7995 betting/gaming, 5967 direct marketing) are always High tier.
Releasing a reserve early requires Head of Merchant Risk approval.
```

9. Save docs/sop-limit-increase.md

docs/sop-limit-increase.md

```
---
doc_type: sop
effective_date: 2026-03-15
owner: Risk Operations
---
# SOP: Processing a Temporary Limit Increase
1. Account manager submits the request with justification (e.g. seasonal sale, flash sale date).
2. Risk Ops checks current utilisation, risk tier, chargeback ratio and watchlist status in the
merchant file.
3. Route to the approver per Merchant Risk Policy §3.
4. Record the approval reference in the limits system; set auto-expiry.
5. Notify the merchant and the account manager.
```

10. Ingest with metadata at three chunk sizes

ingest.py

```
"""Day 5 Lab 1: build Chroma indexes from docs/ with metadata, at three chunk sizes.

Prereq: ollama pull nomic-embed-text.   Run: python ingest.py
Set CHROMA_HOST/CHROMA_PORT to use a Chroma server (Day 9 compose) instead of ./chroma.
"""
import os
import pathlib
import re

import yaml
from langchain_chroma import Chroma
from langchain_core.documents import Document
from langchain_ollama import OllamaEmbeddings
from langchain_text_splitters import MarkdownHeaderTextSplitter, RecursiveCharacterTextSplitter

DOCS_DIR = pathlib.Path(__file__).parent / "docs"
emb = OllamaEmbeddings(model=os.getenv("OLLAMA_EMBED_MODEL", "nomic-embed-text"),
                       base_url=os.getenv("OLLAMA_BASE_URL", "http://localhost:11434"))


def chroma_kwargs() -> dict:
    if os.getenv("CHROMA_HOST"):
        import chromadb
        return {"client": chromadb.HttpClient(host=os.environ["CHROMA_HOST"],
port=int(os.getenv("CHROMA_PORT", "8000")))}
    return {"persist_directory": str(pathlib.Path(__file__).parent / "chroma")}


def load_docs() -> list[Document]:
    out = []
    for p in sorted(DOCS_DIR.glob("*.md")):
        text, meta = p.read_text(), {}
        if m := re.match(r"^---\n(.*?)\n---\n(.*)$", text, re.S):
            meta, text = yaml.safe_load(m.group(1)), m.group(2)
        meta = {k: str(v) for k, v in meta.items()}                # Chroma metadata must be scalar
```

```
        meta["is_current"] = "superseded_by" not in meta           # filterable boolean
        meta["effective_year"] = int(meta.get("effective_date", "1970")[:4])
        out.append(Document(text, metadata={"source": p.stem, **meta}))
    return out


def chunk(docs: list[Document], size: int) -> list[Document]:
    md = MarkdownHeaderTextSplitter([("#", "h1"), ("##", "section")], strip_headers=False)
    rec = RecursiveCharacterTextSplitter(chunk_size=size, chunk_overlap=size // 10)
    chunks = []
    for d in docs:
        for sec in md.split_text(d.page_content):                  # structure-aware first...
            sec.metadata |= d.metadata
            chunks += rec.split_documents([sec])                    # ...then size-bounded
    return chunks


def build(size: int) -> Chroma:
    name = f"northstar_{size}"
    vs = Chroma(collection_name=name, embedding_function=emb, collection_metadata={"hnsw:space":
"cosine"},
                **chroma_kwargs())
    vs.reset_collection()                                          # idempotent re-ingest
    vs.add_documents(chunk(load_docs(), size))
    return vs


def open_store(size: int = 500) -> Chroma:
    return Chroma(collection_name=f"northstar_{size}", embedding_function=emb,
                  collection_metadata={"hnsw:space": "cosine"}, **chroma_kwargs())


if __name__ == "__main__":
    q = "approvals needed for a 12% limit increase for a High-risk-tier merchant"
    for size in (150, 500, 1500):
        vs = build(size)
        print(f"\n=== chunk size {size}: {vs._collection.count()} chunks ===")
        for doc, score in vs.similarity_search_with_relevance_scores(q, k=3):
            print(f"  {score:.3f} {doc.metadata['source']:38} §{doc.metadata.get('section', '-')
[:30]:30} | "
                  f"{doc.page_content[:60]!r}")
```

11. Run the chunk-size experiment

Compare the top-3 chunks for the approval question at 150, 500 and 1500 characters. Which size keeps the approval table whole?

What to look for in the output:

- **150:** ~29 chunks. The table's header row and its data rows land in *different* chunks, so a row like `| 5–15% | High | Head of Merchant Risk |` has no column names next to it.
- **500:** ~10 chunks, usually one per section: the table stays together.
- **1500:** the same 10 here (the documents are short), but on long documents big chunks bury the answer in unrelated text.

RUN
```
python ingest.py
```

12. Explore with a metadata filter

`filter={"is_current": True}` is applied **inside** the vector search, before ranking. Filtering *after* retrieving the top 3 can leave you with zero results.

explore.py

```
# explore.py: look at what the retriever returns
from ingest import open_store

vs = open_store(500)
for doc, score in vs.similarity_search_with_relevance_scores(
        "approvals needed for a 12% limit increase for a High-risk-tier merchant", k=3):
    print(round(score, 3), doc.metadata["source"], doc.metadata.get("section"), "|",
doc.page_content[:80])

# Metadata filter: only current policies (ingest.py sets is_current=False on superseded documents)
retriever = vs.as_retriever(search_kwargs={"k": 4, "filter": {"is_current": True}})
```

13. If something goes wrong

Symptom	Fix
`Connection refused` on port 11434	start Ollama (or point `OLLAMA_BASE_URL` at a shared host)
`model "nomic-embed-text" not found`	`ollama pull nomic-embed-text`
Results look identical for every size	you didn't re-run ingest after changing a document: `reset_collection()` rebuilds it
`Expected metadata value to be a str, int, float or bool`	Chroma metadata must be scalar: lists/dates must be converted (ingest does this)

Expected result. Three collections; at 500 characters the top hit is merchant-risk-policy-2026 §3 (the approval matrix).

LAB BEGINNER 20 MIN

Lab 2: Classic RAG, and how it fails

Goal. Build a textbook RAG chain, then watch it answer confidently and wrongly.

1. Build the chain

No "current only" filter: on purpose.

`classic_rag.py`

```
"""Day 5 Lab 2: textbook RAG chain in LCEL, and its failure modes.

Prereq: python ingest.py.   Run: python classic_rag.py
"""
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

from ingest import open_store
from llm import chat_model

vs = open_store(500)
retriever = vs.as_retriever(search_kwargs={"k": 3})          # no "current only" filter, on purpose


def format_docs(docs) -> str:
    return "\n\n".join(f"[{d.metadata['source']} §{d.metadata.get('section', '-')}]\n{d.page_content}"
for d in docs)


prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer using ONLY the context. Cite sources in [brackets].\n\nContext:\n{context}"),
    ("human", "{question}"),
])
rag = ({"context": retriever | format_docs, "question": RunnablePassthrough()}
       | prompt | chat_model(max_tokens=4000) | StrOutputParser())

QUESTIONS = ["Who approves a 7% limit increase for a Low-risk-tier merchant?",   # should work
             "How long can a limit increase last?",                               # superseded-doc trap
             "What's the limit and who can raise it?",                            # AMBIGUOUS
             "What reserve applies to an iGaming merchant?"]                      # vocabulary gap

if __name__ == "__main__":
    for q in QUESTIONS:
        print("Q:", q, "\nA:", rag.invoke(q), "\n")
```

2. Run the four test questions

RUN `python classic_rag.py`

3. Record the failures

"What's the limit?": mixes API rate limits with processing limits. **"How long can an increase last?"**: may cite the superseded 60 days. **"iGaming reserve"**: vocabulary gap, so the right document isn't found.

4. **Look at what was retrieved**

   Add `print([d.metadata["source"] for d in retriever.invoke(q)])` before each answer. Most RAG bugs are **retrieval** bugs: the model answered faithfully from the wrong chunks.

5. **Discuss**

   What would a careful analyst do differently? (Notice the documents don't match, rephrase, check the date, ask which limit.) That list is the spec for Lab 3.

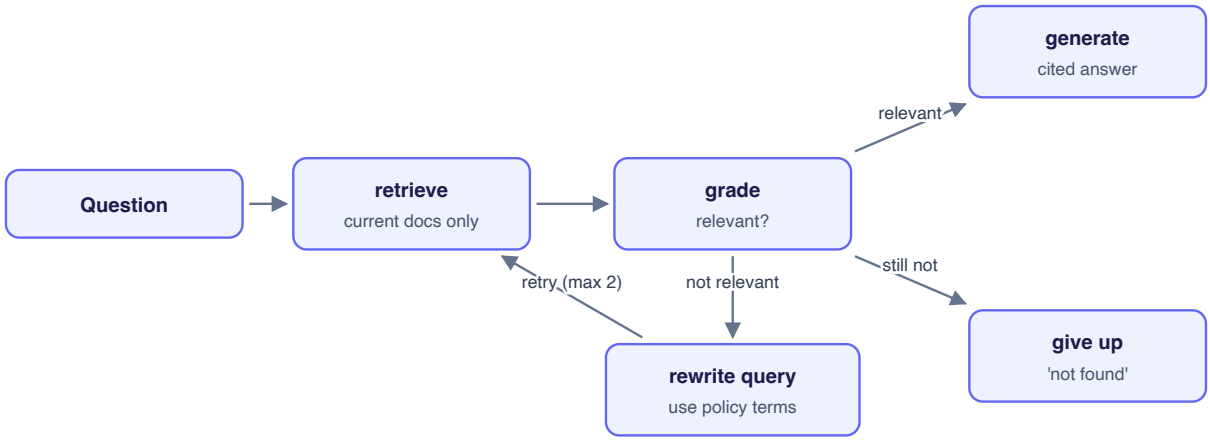
Expected result. At least two wrong or muddled answers, each with a confident tone.

THEORY 10 MIN

Agentic RAG: grade, rewrite, retry, or give up

Put a decision after retrieval instead of trusting it blindly.

Corrective RAG



1. Retrieve (current policies only), then a cheap model GRADES the chunks against the question.
2. Relevant → generate an answer that cites its sources.
3. Not relevant → rewrite the query in Northstar's vocabulary ('iGaming' → 'MCC 7995 betting/gaming, High tier').
4. Retrieve again, at most twice.
5. Still nothing? Say so honestly instead of guessing. 'I couldn't find it' beats a wrong policy.

RAG, fine-tuning or a long context window?

Option	Use when	Avoid when
Long context (Claude: 1M tokens)	small corpus, changes often	large corpus, high volume, per-user access control
RAG	large or changing corpus, need citations and access control	answers need the whole corpus at once
Fine-tuning	teach **style or a narrow skill**	teaching **facts that change** (limits, rates, policies)

Rule of thumb: facts that change → retrieve them. Behaviour → prompt or fine-tune.

LAB INTERMEDIATE 35 MIN

Lab 3: Agentic RAG in LangGraph

Goal. Retrieval that grades itself, rewrites bad queries, and plugs into Meridian as a tool.

1. Read the graph

Find the five nodes, the `decide` function and the retry cap. `search_policies` at the bottom wraps the whole graph as a tool.

agentic_rag.py

```
"""Day 5 Lab 3: Agentic (corrective) RAG in LangGraph: retrieve -> grade -> rewrite/re-retrieve ->
generate | give up.

Prereq: python ingest.py.   Run: python agentic_rag.py
"""
from typing import Literal

from langchain_core.documents import Document
from langchain_core.tools import tool
from langgraph.graph import END, START, StateGraph
from pydantic import BaseModel, Field
from typing_extensions import TypedDict

from classic_rag import format_docs
from ingest import open_store
from llm import MODEL_FAST, chat_model

llm = chat_model(max_tokens=8000)
grader_llm = chat_model(MODEL_FAST, max_tokens=1000)          # cheap, narrow job (or a local
ChatOllama)
retriever = open_store(500).as_retriever(search_kwargs={"k": 4, "filter": {"is_current": True}})
MAX_RETRIES = 2


class Grade(BaseModel):
    relevant: bool = Field(description="Do these chunks contain enough to answer the question?")
    missing: str = Field(description="What information is missing or ambiguous, if anything")


class Rewrite(BaseModel):
    query: str = Field(description="A better search query using Northstar's policy vocabulary")


class RAGState(TypedDict):
    question: str
    query: str
    docs: list[Document]
    retries: int
    grade: Grade | None
    answer: str


def retrieve(s: RAGState):
    return {"docs": retriever.invoke(s["query"])}


def grade_documents(s: RAGState):
    g = grader_llm.with_structured_output(Grade).invoke(
        f"Question: {s['question']}\n\nRetrieved chunks:\n{format_docs(s['docs'])}\n\n"
        "Be strict: chunks about API RATE limits or REFUND limits do not answer questions about
```

```
merchant PROCESSING "
        "limits, and vice versa. If the question is ambiguous about which limit, say so in
'missing'.")
    return {"grade": g}


def decide(s: RAGState) -> Literal["generate", "rewrite", "give_up"]:
    if s["grade"].relevant:
        return "generate"
    return "rewrite" if s["retries"] < MAX_RETRIES else "give_up"


def rewrite(s: RAGState):
    r = llm.with_structured_output(Rewrite).invoke(
        f"Original question: {s['question']}\nPrevious query: {s['query']}\nWhat was missing:
{s['grade'].missing}\n"
        "Rewrite the search query. Use internal terms like 'temporary processing limit increase',
'High risk tier', "
        "'MCC 7995 betting/gaming', 'rolling reserve', 'chargeback watchlist'.")
    print(f"  rewrite #{s['retries'] + 1}: {r.query}")
    return {"query": r.query, "retries": s["retries"] + 1}


def generate(s: RAGState):
    msg = llm.invoke(f"Answer using ONLY these sources and cite them in
[brackets].\n\n{format_docs(s['docs'])}\n\n"
                     f"Question: {s['question']}")
    return {"answer": msg.text}


def give_up(s: RAGState):
    return {"answer": f"I couldn't find a reliable source for this in Northstar's documents (last
search: "
                      f"'{s['query']}'). Missing: {s['grade'].missing}. Please clarify or contact
Merchant Risk."}


b = StateGraph(RAGState)
for name, fn in [("retrieve", retrieve), ("grade_documents", grade_documents), ("rewrite", rewrite),
                 ("generate", generate), ("give_up", give_up)]:
    b.add_node(name, fn)
b.add_edge(START, "retrieve")
b.add_edge("retrieve", "grade_documents")
b.add_conditional_edges("grade_documents", decide)
b.add_edge("rewrite", "retrieve")
b.add_edge("generate", END)
b.add_edge("give_up", END)
rag_graph = b.compile()


def ask(question: str) -> dict:
    return rag_graph.invoke({"question": question, "query": question, "docs": [], "retries": 0,
                             "grade": None, "answer": ""})


@tool
def search_policies(question: str) -> str:
    """Search Northstar Pay's internal merchant risk policies, SOPs, chargeback & reserve rules, API
limits and pricing.
    Returns a cited answer or an explicit 'not found'. Superseded policies are excluded."""
    return ask(question)["answer"]


if __name__ == "__main__":
    for q in ["What's the limit and who can raise it?", "What reserve applies to an iGaming
```

```
merchant?",
              "How long can a limit increase last?"]:
        out = ask(q)
        print(f"Q: {q}\n  retries: {out['retries']}\n  -> {out['answer'][:400]}\n")

    # Plug into Meridian: the agent chooses when to read policy
    from meridian import build_agent
    from meridian import ALL_TOOLS
    agent = build_agent(tools=ALL_TOOLS + [search_policies])
    res = agent.invoke({"messages": [("user", "CRUX needs a 12% processing limit increase. Who must
approve it?")]},
                       {"recursion_limit": 25})
    print("Meridian:", res["messages"][-1].text)
```

2. Re-run the questions that fooled classic RAG

run

```
# Try the questions that fooled classic RAG
from agentic_rag import ask

for q in ["What's the limit and who can raise it?",
          "What reserve applies to an iGaming merchant?",
          "How long can a limit increase last?"]:
    out = ask(q)
    print(f"Q: {q}\n  retries: {out['retries']}\n  -> {out['answer'][:400]}\n")
```

3. Let Meridian choose when to read policy

The bottom of the file asks Meridian: **"CRUX needs a 12% increase. Who must approve it?"** It should look up CRUX (High tier, watchlist) **and** search the policy.

RUN `python agentic_rag.py`

4. Compare with Lab 2

For each failure from Lab 2: correct now? How many retries? How many more tokens?

5. Trace a retry

Run the iGaming question with `stream_mode="updates"` and print each node. You should see `retrieve → grade_documents (relevant=False) → rewrite → retrieve → grade_documents (relevant=True) → generate`. The rewritten query uses the documents' words (*MCC 7995*, *High tier*): the vocabulary gap closed.

6. Tune the grader

The grader is a cheap model with a strict prompt. Remove the sentence about API rate limits and refund limits from its prompt and re-run "*What's the limit?*". Does it still notice the ambiguity? Graders need **examples of what 'irrelevant' means in your domain**.

Expected result. The ambiguous question gets a clarifying answer or an honest 'not found'; CRUX → **not eligible** (on the chargeback watchlist).

THEORY ADVANCED 10 MIN

Measure retrieval before you tune it

Every RAG change (chunk size, filters, hybrid search, a new embedding model) should move a number you trust.

Build a **golden set**: questions paired with the document(s) that must be retrieved. Then score each retriever setting:

Metric	Meaning	Use it when
hit@k	share of questions with a right document in the top k	the prompt gets k chunks
hit@1	the right document is ranked *first*	small corpora (top-3 of 7 documents is too easy)
MRR	average of 1 / rank of the first right document	comparing rankings in one number
stale@k	a superseded document appears in the top k	you keep old versions around

Retrieval metrics need **no LLM**: they're fast, free and repeatable, so run them on every change, in CI (Day 8). Answer-level metrics (faithfulness, answer correctness) come next, and do need a judge model.

Key points

- Start with 10–20 real questions; add one every time retrieval fails in use.
- Separate retrieval quality from answer quality: fix the first before blaming the model.

LAB ADVANCED 20 MIN

Lab 4: Score your retriever

No LLM needed: just the embeddings from Lab 1.

Goal. Score 6 retriever settings (3 chunk sizes × filter on/off) on a 12-question golden set, and read the misses.

1. The golden set

Three questions are traps: the 2024 policy says 60 days (it's 30 now), 'betting' must find 'MCC 7995', and the chargeback **fee** lives in the pricing sheet, not the chargeback policy.

golden.py

```
# golden.py — questions with the document that MUST be retrieved to answer them correctly.
# Small on purpose: 12 good questions beat 500 vague ones. Add a case every time retrieval fails in real use.
GOLDEN = [
    ("Who approves a 10% limit increase for a High-tier merchant?", ["merchant-risk-policy-2026"]),
    ("How long can a temporary limit increase last?", ["merchant-risk-policy-2026"]),          # 2024 policy says 60 days: a trap
    ("What happens when a merchant goes above 100% of its limit?", ["merchant-risk-policy-2026"]),
    ("What reserve do we hold for a betting merchant?", ["rolling-reserve-rules"]),            # 'betting' vs 'MCC 7995'
    ("A merchant has a 1% chargeback ratio. What's its status?", ["chargeback-and-disputes-policy"]),
    ("How many days do we have to send dispute evidence?", ["chargeback-and-disputes-policy"]),
    ("How many refund requests per second can a merchant make?", ["api-rate-limits"]),
    ("What is the MDR on a domestic credit card payment?", ["pricing-sheet-2026-q4"]),
    ("What do we do after a limit increase is approved?", ["sop-limit-increase"]),
    ("Which FX rate is used to convert month-to-date volume?", ["merchant-risk-policy-2026"]),
    ("How much do we charge a merchant per chargeback?", ["pricing-sheet-2026-q4"]),          # chargeback fee lives in PRICING
    ("Can a merchant on the chargeback watchlist get a limit increase?", ["merchant-risk-policy-2026", "chargeback-and-disputes-policy"]),
]
```

2. The evaluator

eval_retrieval.py

```
# eval_retrieval.py — measure retrieval before you tune it. No LLM needed: just embeddings (Ollama).
# Prereq: python ingest.py (builds the 150 / 500 / 1500 collections).   Run:  python eval_retrieval.py
from golden import GOLDEN
from ingest import open_store

STALE = "merchant-risk-policy-2024-SUPERSEDED"

def score(search, k: int = 3):
    """hit@1: the right document is the FIRST result (what a 1-chunk prompt would see).
    MRR: mean of 1/rank of the first right document within the top k (1.0 = always first).
    stale@k: share of questions where the superseded policy is in the top k (a wrong answer waiting to happen)."""
    hit1, rr, stale, misses = 0, 0.0, 0, []
    for question, expected in GOLDEN:
        sources = [d.metadata["source"] for d in search(question, k)]
        rank = next((i + 1 for i, s in enumerate(sources) if s in expected), None)
        hit1 += rank == 1
        rr += 1 / rank if rank else 0
        stale += STALE in sources
```

```
        if rank != 1:
            misses.append((question, sources))
    n = len(GOLDEN)
    return {"hit@1": hit1 / n, "MRR": rr / n, "stale@3": stale / n}, misses

if __name__ == "__main__":
    print(f"{'chunks':>6} {'filter':>12} {'hit@1':>6} {'MRR':>5} {'stale@3':>8}")
    shown = None
    for size in (150, 500, 1500):
        store = open_store(size)
        for current_only in (False, True):
            flt = {"is_current": True} if current_only else None
            m, misses = score(lambda q, k: store.similarity_search(q, k=k, filter=flt))
            print(f"{size:>6} {'current only' if current_only else 'none':>12} {m['hit@1']:>6.0%}
{m['MRR']:>5.2f} {m['stale@3']:>8.0%}")
            if size == 500 and current_only:
                shown = misses
    print("\nNot ranked first (500 chars, current only):")
    for question, got in shown or []:
        print(f"  ✗ {question}\n      top 3: {got}")
```

RUN `python eval_retrieval.py`

3. Read the table

Measured with nomic-embed-text (your numbers may differ slightly):

chunks	filter	hit@1	MRR	stale@3
150	none	83%	0.92	25%
150	current only	83%	0.92	0%
500	none	92%	0.94	**50%**
500	current only	92%	0.96	**0%**

Without the filter, the superseded policy is in the top 3 for up to **half** the questions: each one a wrong answer waiting to happen.

4. Read the misses

"How long can a temporary limit increase last?" ranks the SOP first, not the policy. Is that wrong? The SOP mentions increases but not their length. A grader (Lab 3) or a reranker would catch it; a bigger k would only hide it.

5. Add your own case

Write a question you think will fail (e.g. "*What's the cross-border fee on a wallet payment?*"), add it with its expected source, and run again. A golden set grows from real failures.

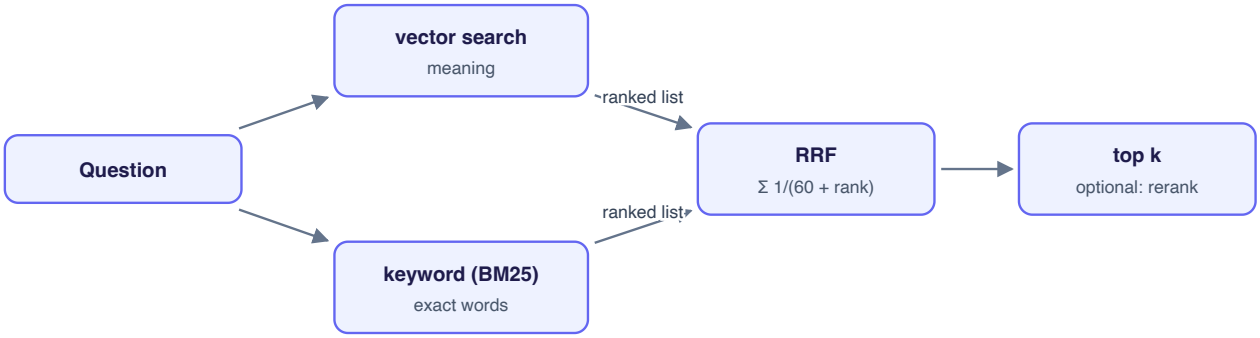
Expected result. A table of 6 settings where the 'current only' filter takes stale@3 to 0%, and 500-character chunks rank the right document first more often than 150.

THEORY ADVANCED 10 MIN

Hybrid search, fusion and reranking

Vectors find meaning ('betting' ≈ 'gaming'); keywords find exact tokens ('MDR', '7995', 'HTTP 429'). Payments text needs both.

Hybrid retrieval with Reciprocal Rank Fusion



1. Run both searches over the same chunks.
2. RRF adds 1/(60 + rank) for each list a chunk appears in. Ranks, not raw scores: cosine and BM25 scores aren't comparable.
3. Chunks that rank well in either list rise. A cross-encoder or LLM reranker can reorder the final few.

- **Reranking:** retrieve 20–50 candidates cheaply, then score each (question, chunk) pair with a stronger model and keep the best 3–5. Cross-encoders (e.g. a BGE or Cohere reranker) are fast; an LLM reranker is flexible but costs a call.
- **Contextual chunk headers:** prepend the document title and section to each chunk before indexing. Often helps embeddings; **measure it**: in Lab 5 it made keyword search *worse* on our corpus.
- **Query rewriting:** Lab 3's rewrite node, or generate 3 phrasings and fuse their results (multi-query).

LAB ADVANCED 25 MIN

Lab 5: Hybrid search, measured

Goal. Add BM25 keyword search over the same chunks, fuse it with vector search, and beat both on the golden set.

1. Install

RUN `pip install rank-bm25`

2. Read and run

hybrid.py

```python
# hybrid.py — keyword search (BM25) + vector search, merged with Reciprocal Rank Fusion. Then measure it.
# Prereq: python ingest.py, pip install rank-bm25.   Run:  python hybrid.py
import re
from langchain_core.documents import Document
from rank_bm25 import BM25Okapi
from eval_retrieval import score
from ingest import open_store

FILTER = {"is_current": True}
tokenize = lambda text: re.findall(r"[a-z0-9]+", text.lower())

def with_context(chunk: Document) -> str:
    """Contextual chunk header: the chunk's document title and section, prepended before indexing."""
    return f"{chunk.metadata.get('h1', chunk.metadata['source'])} › {chunk.metadata.get('section', '')}\n{chunk.page_content}"

def build(size: int = 150, contextual: bool = False):
    """Three search functions over the SAME chunks (one Chroma collection): vector, keyword and hybrid."""
    store = open_store(size)
    raw = store.get(where=FILTER)
    chunks = [Document(text, metadata=meta) for text, meta in zip(raw["documents"], raw["metadatas"])]
    bm25 = BM25Okapi([tokenize(with_context(c) if contextual else c.page_content) for c in chunks])

    def vector_search(question: str, k: int) -> list[Document]:
        return store.similarity_search(question, k=k, filter=FILTER)

    def keyword_search(question: str, k: int) -> list[Document]:   # exact words: 'MDR', '7995', '429', 'four-eyes'
        scores = bm25.get_scores(tokenize(question))
        return [chunks[i] for i in sorted(range(len(chunks)), key=lambda i: -scores[i])[:k]]

    def hybrid_search(question: str, k: int, depth: int = 10, c: int = 60) -> list[Document]:
        """Reciprocal Rank Fusion: score = sum of 1 / (c + rank) over both lists. It uses ranks, not raw scores,
        so two scoring scales that aren't comparable (cosine vs BM25) combine with no tuning."""
        fused: dict[str, tuple[float, Document]] = {}
        for results in (vector_search(question, depth), keyword_search(question, depth)):
            for rank, doc in enumerate(results, start=1):
                prev = fused.get(doc.page_content, (0.0, doc))[0]
                fused[doc.page_content] = (prev + 1 / (c + rank), doc)
        return [doc for _, doc in sorted(fused.values(), key=lambda x: -x[0])[:k]]

    return vector_search, keyword_search, hybrid_search

if __name__ == "__main__":
```

```
    vector, keyword, hybrid = build(150)
    print(f"{'method':>8} {'hit@1':>6} {'MRR':>5}")
    for name, search in [("vector", vector), ("keyword", keyword), ("hybrid", hybrid)]:
        m, misses = score(search)
        print(f"{name:>8} {m['hit@1']:>6.0%} {m['MRR']:>5.2f}   misses: {[q[:38] for q, _ in
misses]}")
```

RUN `python hybrid.py`

3. **Read the scores**

   Measured on 150-character chunks:

method	hit@1	MRR
vector	83%	0.92
keyword	75%	0.86
hybrid	**92%**	**0.96**

   Each method misses different questions, which is exactly when fusion helps.

4. **Try contextual headers**

   Call `build(150, contextual=True)` and score the **keyword** search. On our corpus it drops from 75% to 67%: the title words (*Merchant Risk Policy*) now appear in every chunk of that document and blur BM25. Re-embed with headers too and score the vector search: does it help there? Never assume; measure.

5. **Stretch: an LLM reranker**

   Take the hybrid top 10, ask the fast model to score each chunk 0–3 for the question (structured output), keep the top 3, and re-score. Compare hit@1 *and* the extra latency and tokens.

Expected result. hybrid ranks the right document first for 11 of 12 questions, ahead of vector (10) and keyword (9) alone.

THEORY ADVANCED 8 MIN

Grounded answers you can check

"Cite sources" in a prompt is a request, not a guarantee. Make it checkable:

1. **Number the sources** you give the model, and ask for citation numbers in **structured output**.
2. **Check in code** that every citation points at a source you actually provided, and reject answers with none.
3. **Ask a judge** (a second, cheaper model) whether each claim is supported by the cited text. Give the judge the **question** too: a short answer like "Head of Merchant Risk" is only checkable in context.
4. **Allow "NOT FOUND".** An honest refusal beats a fluent guess, especially about money and compliance.

Judges make mistakes too. Measure the judge against a few hand-labelled answers before trusting its verdicts (Day 8).

LAB ADVANCED 25 MIN

Lab 6: Answers with verified citations

Goal. Answer from hybrid retrieval with numbered citations, verify them in code, and grade groundedness with a judge model.

1. Read and run

`grounded.py`

```
# grounded.py — answers you can check: numbered sources, citations verified in code, and a
groundedness grader.
# Prereq: python ingest.py.   Run:  python grounded.py
from pydantic import BaseModel, Field
from hybrid import build
from llm import MODEL_FAST, chat_model

class CitedAnswer(BaseModel):
    answer: str = Field(description="The answer, or exactly 'NOT FOUND' if the sources don't contain
it")
    citations: list[int] = Field(description="Numbers of the sources that support the answer, e.g. [1,
3]")

class Groundedness(BaseModel):
    grounded: bool = Field(description="True only if EVERY claim in the answer is stated in the cited
sources")
    unsupported: list[str] = Field(description="Claims in the answer that the sources don't support")

# 500-character chunks: big enough to keep a table row with its header (150 splits them; see Lab 4).
_, _, hybrid_search = build(500)

writer = chat_model(max_tokens=2000).with_structured_output(CitedAnswer)
judge = chat_model(MODEL_FAST, max_tokens=1000).with_structured_output(Groundedness)   # a different,
cheaper model

def ask(question: str) -> dict:
    docs = hybrid_search(question, k=4)
    sources = "\n\n".join(f"[{i}] ({d.metadata['source']})\n{d.page_content}" for i, d in
enumerate(docs, start=1))
    out = writer.invoke(
        "Answer the question using ONLY the numbered sources. Cite the numbers you used. "
        "If the sources don't contain the answer, reply exactly NOT FOUND with no citations.\n\n"
        f"{sources}\n\nQuestion: {question}")

    # 1. Check in CODE: every citation must point at a source we actually gave the model.
    valid = [c for c in out.citations if 1 <= c <= len(docs)]
    if out.answer.strip() == "NOT FOUND":
        return {"answer": "I couldn't find this in Northstar's documents.", "status": "not found",
"sources": []}
    if not valid:
        return {"answer": out.answer, "status": "REJECTED: no valid citations", "sources": []}

    # 2. Check with a JUDGE: is each claim supported by the cited text?
    cited = "\n\n".join(f"[{c}] {docs[c - 1].page_content}" for c in valid)
    # Give the judge the QUESTION too: a short answer like "Head of Merchant Risk" is only checkable
in context.
    verdict = judge.invoke(f"Sources:\n{cited}\n\nQuestion: {question}\nAnswer: {out.answer}\n\n"
                           "As an answer to this question, is every claim supported by the sources?")
    status = "grounded" if verdict.grounded else f"NOT GROUNDED: {verdict.unsupported}"
    return {"answer": out.answer, "status": status, "sources": sorted({docs[c - 1].metadata["source"]
for c in valid})}
```

```
if __name__ == "__main__":
    for q in ["Who approves a 10% limit increase for a High-tier merchant?",
              "How long can a temporary limit increase last?",
              "What is Northstar's limit for UPI refunds in Japan?"]:          # not in any document
        r = ask(q)
        print(f"Q: {q}\nA: {r['answer']}\n   [{r['status']}] {r['sources']}\n")
```

RUN `python grounded.py`

2. Check the three outcomes

Question	Expected
Who approves a 10% increase for a High-tier merchant?	*Head of Merchant Risk*, **grounded**, merchant-risk-policy-2026
How long can an increase last?	*30 calendar days*, **grounded** (not the superseded 60)
UPI refund limit in Japan?	**not found**: no document covers it

3. Make it fail on purpose

Change `build(500)` to `build(150)` and ask the approval question again. The table row and its header are now in different chunks: you'll get *not found*, or an answer the judge flags. Smaller isn't always better.

4. Remove the question from the judge

Delete `Question: {question}` from the judge prompt and re-run. A correct short answer can now be marked NOT GROUNDED: the judge can't see why 'Head of Merchant Risk' answers *this* question.

5. Stretch: show citations to users

Return `sources` with section names and render them under the answer (e.g. *merchant-risk-policy-2026 § 3*). Analysts trust answers they can verify in one click.

Expected result. Two grounded answers citing merchant-risk-policy-2026 and one 'not found'; no answer cites the superseded policy.

THEORY ADVANCED 5 MIN

RAG in production

Concern	Do this
Who may see what	filter by tenant / role **inside** the search (`filter={"team": ...}`), from the user's identity, never from the prompt
Changing documents	re-ingest on change with stable IDs; keep `effective_date` and `superseded_by`; filter to current
Injected instructions	documents can contain "ignore your rules…": treat retrieved text as data, never instructions (Day 9)
Quality drift	golden-set scores in CI (Day 8); alert when hit@1 drops
Cost and latency	cache embeddings; small models for grading and rewriting; cap retries

Concern	Do this
Relationships across documents	GraphRAG (entities + links) when answers need multi-hop reasoning; it costs far more to build

QUIZ 5 MIN

Check yourself

1. **Cosine vs dot product on normalised vectors?**
   A. Opposite rankings
   B. The same ranking
   C. Dot product is always worse
   D. They can't be compared

2. **Why does chunk size matter if nothing errors?**
   A. It doesn't
   B. It silently changes what's retrieved, and so answer quality
   C. It changes the model
   D. It changes cost only

3. **What does the grade node add over classic RAG?**
   A. Speed
   B. A decision: re-retrieve, rewrite, or give up instead of answering from bad context
   C. Cheaper embeddings
   D. More documents

4. **Should you fine-tune to teach the new rolling-reserve percentages?**
   A. Yes
   B. No: changing facts belong in retrieval

5. **How do you stop a superseded policy from answering?**
   A. A bigger model
   B. A metadata filter (only current documents)
   C. A longer prompt
   D. Smaller chunks

6. **Why does hybrid (vector + BM25) beat either alone?**
   A. It uses a bigger model
   B. They miss different questions: meaning vs exact words
   C. BM25 is always better
   D. It retrieves more chunks

7. **What does Reciprocal Rank Fusion combine?**
   A. Raw similarity scores
   B. Ranks from each list: Σ 1/(c + rank)
   C. Embeddings
   D. Token counts

8. **Your judge marks 'Head of Merchant Risk' as NOT GROUNDED even though the table says so. Likely cause?**
   A. The model is wrong
   B. The judge wasn't given the question, so it can't see what the answer refers to
   C. The chunk is too big
   D. Temperature

9. **Why measure retrieval with hit@1 and MRR before changing prompts?**
    A. They're required
    B. Most RAG errors are retrieval errors, and these metrics are fast, cheap and need no LLM
    C. Prompts don't matter
    D. To use more tokens

RECAP 3 MIN

Day 5 recap

- Embeddings capture meaning as used in training data, not identifiers, negation or your jargon.
- Chunk by structure, tag with metadata, filter to current documents.
- Classic RAG answers confidently from whatever it retrieved.
- Agentic RAG grades, rewrites and retries, or honestly gives up.
- Facts that change → retrieval; behaviour → prompt or fine-tune.
- Measure retrieval (hit@1, MRR, stale@k) on a golden set; hybrid search + fusion beat either method alone; verify citations in code and with a judge.

Next: Day 6: the knowledge base becomes an MCP server, so any agent in any language can use it.

Quiz answer key

1. **B. The same ranking**
   On normalised vectors they're equivalent.
2. **B. It silently changes what's retrieved, and so answer quality**
   Too small loses context; too big buries the answer.
3. **B. A decision: re-retrieve, rewrite, or give up instead of answering from bad context**
   Classic RAG trusts every retrieval; agentic RAG checks it.
4. **B. No: changing facts belong in retrieval**
   Fine-tuned facts go stale and can't be cited.
5. **B. A metadata filter (only current documents)**
   Tag documents at ingest and filter at query time.
6. **B. They miss different questions: meaning vs exact words**
   Fusion helps exactly when the two lists disagree.
7. **B. Ranks from each list: Σ 1/(c + rank)**
   Ranks are comparable across methods; cosine and BM25 scores aren't.
8. **B. The judge wasn't given the question, so it can't see what the answer refers to**
   Short answers are only checkable in the context of the question.
9. **B. Most RAG errors are retrieval errors, and these metrics are fast, cheap and need no LLM**
   Fix what was retrieved before blaming the answer.